Sprint 2 corregido. René Balderrama A.

Descripción

Repasar las nociones básicas para el uso de la base de datos relacionales. En este sprint, iniciarás tu experiencia práctica con una base de datos que contiene información de una empresa dedicada a la venta de productos online. En esta actividad, te enfocarás en datos relacionados con las transacciones efectuadas y la información corporativa de las empresas que participaron.

Importante

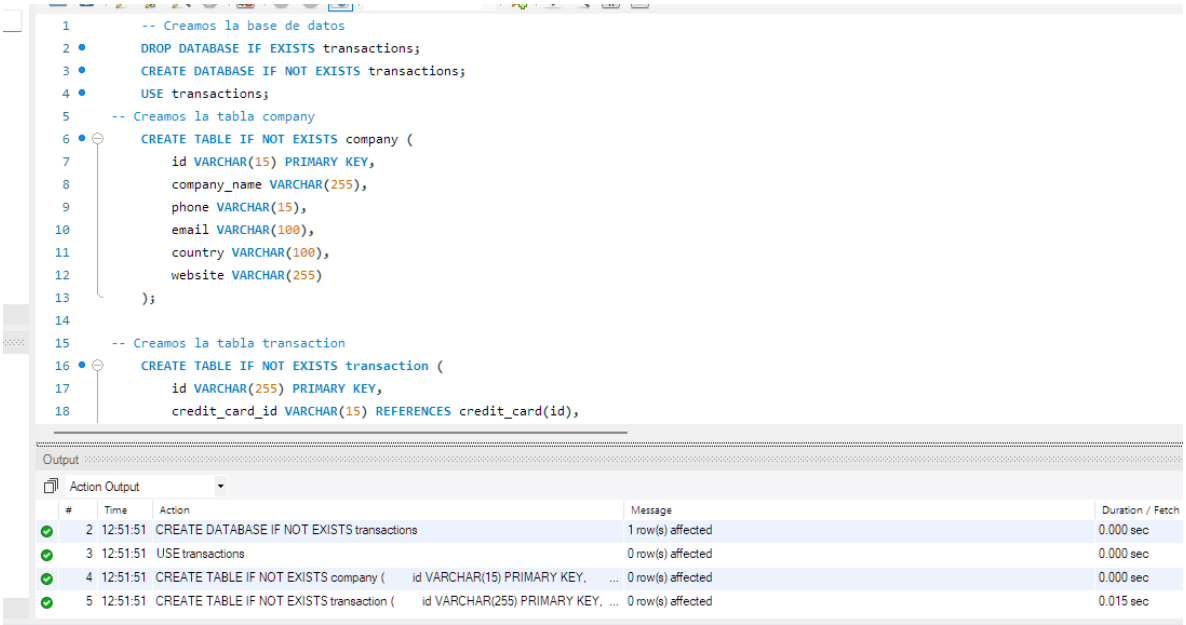
Todas las transformaciones e importaciones que se te piden en esta tarea deben realizarse utilizando código SQL. NO SE PERMITE realizar los cambios utilizando el wizard.

Nivel 1

Ejercicio 1

A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

Creación de tablas:



```sql
-- Creamos la base de datos
DROP DATABASE IF EXISTS transactions;
CREATE DATABASE IF NOT EXISTS transactions;
USE transactions;
-- Creamos la tabla company
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);

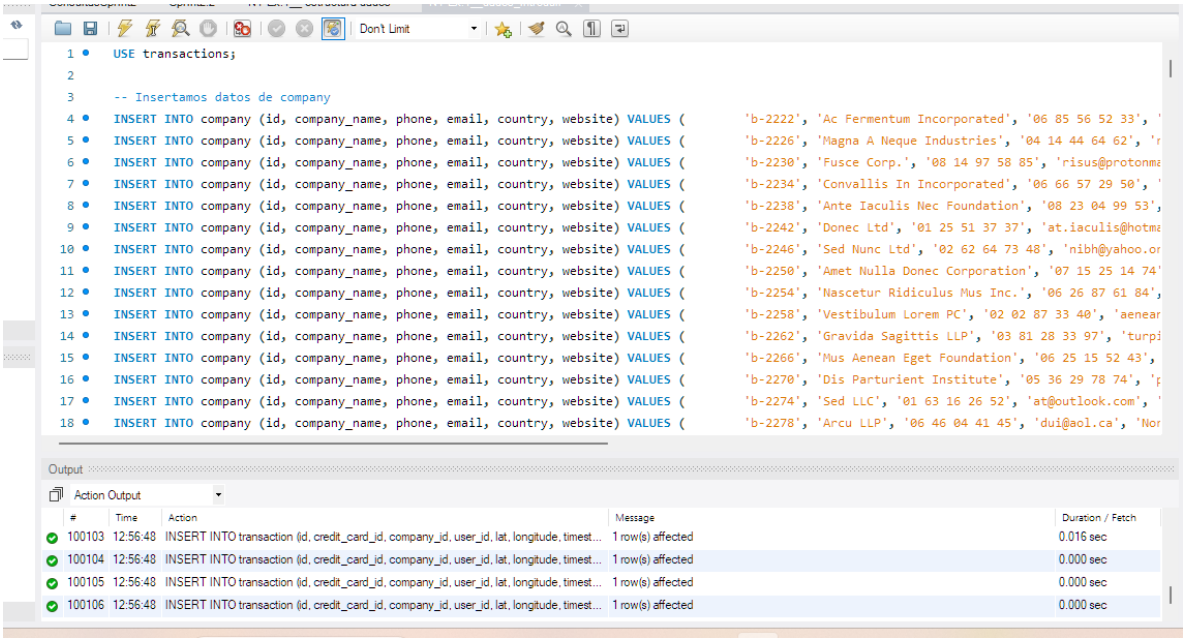
-- Creamos la tabla transaction
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_card(id),
```

Output

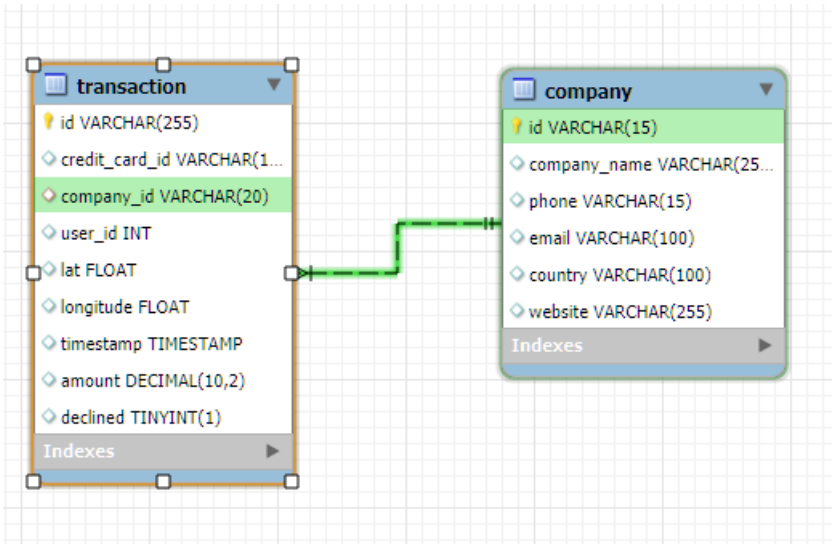
Action Output

#	Time	Action	Message	Duration / Fetch
2	12:51:51	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.000 sec
3	12:51:51	USE transactions	0 row(s) affected	0.000 sec
4	12:51:51	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.000 sec
5	12:51:51	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, ...	0 row(s) affected	0.015 sec

Inserción de datos:



Se ha creado la base de datos transactions a partir de los scripts proporcionados para el ejercicio. El esquema está compuesto por dos tablas principales: company transaction.



Las tablas están relacionadas mediante una relación 1:N (uno a muchos). Una compañía puede estar asociada a múltiples transacciones, mientras que cada transacción está asociada a una única compañía.

transaction.company_id → company.id

donde company.id es la clave primaria de la tabla company y transaction.company_id actúa como clave foránea en la tabla transaction.

La tabla company almacena la información relativa a las empresas o comercios en los que se realizan las transacciones.

La tabla company contiene la información relacionada con las empresas o comercios.

Está formada por las siguientes variables: id, de tipo VARCHAR(15), que identifica de forma única a cada compañía y actúa como clave primaria; company_name, de tipo VARCHAR(255), que corresponde al nombre de la compañía; phone, de tipo VARCHAR(15), que almacena su número de teléfono; email, de tipo VARCHAR(100), que contiene su dirección de correo electrónico; country, de tipo VARCHAR(100), que indica el país donde se encuentra la compañía; y website, de tipo VARCHAR(255), que almacena la página web de la compañía.

La tabla transaction: es la tabla principal de operaciones/transacciones.

Contiene la información relacionada con las operaciones realizadas. Está formada por las siguientes variables: id, de tipo VARCHAR(255), que identifica de forma única cada transacción y actúa como clave primaria; credit_card_id, de tipo VARCHAR(...), que identifica la tarjeta utilizada; company_id, de tipo VARCHAR(20), que identifica la compañía asociada y actúa como clave foránea que referencia a company.id; user_id, de tipo INT, que identifica al usuario que realiza la operación; lat, de tipo FLOAT, que indica la latitud geográfica de la operación; longitude, de tipo FLOAT, que indica la longitud geográfica; timestamp, de tipo TIMESTAMP, que registra la fecha y hora en la que se realizó la transacción; amount, de tipo DECIMAL(10,2), que indica el importe económico de la operación; y declined, de tipo TINYINT(1), que indica si la transacción fue rechazada.

Ejercicio 2: Utilizando JOIN realizarás las siguientes consultas: Listado de los países que están generando ventas.

```sql
-- Ejercicio 2
-- Listado de países que estam gemeramdp vemtas
SELECT c.country, COUNT(t.id) AS total_ventas
FROM company c
JOIN transaction t ON c.id = t.company_id
WHERE t.declined = 0
GROUP BY c.country
ORDER BY total ventas DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	total_ventas
Sweden	16484
Netherlands	15033
United Kingdom	13754
Italy	13643
Germany	13269
France	4695
United States	4011
Belgium	3515
Norway	3011
Ireland	2651

Result 3

Output

Action Output

#	Time	Action	Message
1	13:33:58	SELECT c.country, COUNT(t.id) AS total_ventas FROM company c JOIN transaction t O...	15 row(s) returned

Desde cuántos países se generan las ventas.

```sql
-- Desde cuántos países se generan las ventas.
SELECT COUNT(DISTINCT c.country) AS total_paises_con_ventas
FROM company c
JOIN transaction t ON c.id = t.company_id
WHERE t.declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_paises_con_ventas
15

Result 4

Output

Action Output

#	Time	Action	Message
1	13:40:20	SELECT COUNT(DISTINCT c.country) AS total_paises_con_ventas FROM company c J...	1 row(s) returned

Identifica a la compañía con la mayor media de ventas.

```sql
-- Identifica a la compañía con la mayor media de ventas.
SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas
FROM company c
JOIN transaction t ON c.id = t.company_id
WHERE t.declined = 0
GROUP BY c.id
ORDER BY media_ventas DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	company_name	media_ventas
b-2222	Ac Fermentum Incorporated	284.91

Result 5

Output

Action Output

#	Time	Action	Message
1	13:45:49	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM co...	1 row(s) returned

Ejercicio 3

Utilizando sólo subconsultas (sin utilizar JOIN):

Muestra todas las transacciones realizadas por empresas de Alemania.

```sql
-- Ejercicio 3
SELECT *
FROM transaction t
WHERE EXISTS (
    SELECT 1
    FROM company c
    WHERE c.id = t.company_id
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0
00687139-48B2-4FFA-8E73-B20376F04AB4	CcS-4870	b-2222	289	51.1966	10.4669	2019-03-09 19:37:49	524.84	0
0074F4DD-32F1-4827-8758-55896314623A	CcS-8081	b-2222	3500	39.7016	-8.50325	2016-12-26 23:06:57	491.90	0
00AAB9CD-39D6-4DCB-8A1D-13BE73DC90A9	CcS-6797	b-2222	2216	55.7652	-3.76245	2021-04-25 03:06:59	167.15	0
00BE09D4-6920-47D8-ABE8-325E2269829D	CcS-4983	b-2222	402	38.708	-9.12993	2019-02-27 15:25:16	141.66	0
00DA0383-E048-4577-8ED1-3C56C258FF2F	CcS-9223	b-2222	4642	51.1742	10.2027	2019-03-21 11:47:34	325.62	0

transaction 6

Output

Action Output

#	Time	Action	Message
1	13:52:19	SELECT * FROM transaction t WHERE EXISTS (SELECT 1 FROM company c ...	13291 row(s) returned

Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```sql
-- Lista las empresas que han realizado transacciones
-- por un amount superior a la media de todas las transacciones.
SELECT c.id, c.company_name
FROM company c
WHERE EXISTS (
    SELECT 1
    FROM transaction t
    WHERE t.company_id = c.id
      AND t.amount > (SELECT AVG(amount) FROM transaction));
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.
b-2234	Convallis In Incorporated
b-2238	Ante Iaculis Nec Foundation

company 7

Output

Action Output

#	Time	Action	Message
1	13:57:35	SELECT c.id, c.company_name FROM company c WHERE EXISTS (SELECT 1 F...	100 row(s) returned

Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

```sql
-- Eliminarán del sistema las empresas que carecen de transacciones registradas,
-- entrega el listado de estas empresas.
SELECT c.id, c.company_name, c.country
FROM company c
WHERE NOT EXISTS (
    SELECT 1
    FROM transaction t
    WHERE t.company_id = c.id);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	country
NULL	NULL	NULL

company 8

Output

Action Output

#	Time	Action	Message
1	14:05:30	SELECT c.id, c.company_name, c.country FROM company c WHERE NOT EXISTS (...	0 row(s) returned

Ejercicio 4

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

```sql
-- Ejercicio 4 Creación de tabla credit_card
CREATE TABLE credit_card (
    id VARCHAR(20) PRIMARY KEY,
    iban VARCHAR(50) NOT NULL,
    pan VARCHAR(30) NOT NULL,
    pin VARCHAR(10) NOT NULL,
    cvv VARCHAR(10) NOT NULL,
    expiring_date VARCHAR(20)
);
```

Output

Action Output

#	Time	Action	Message
1	14:34:53	CREATE TABLE credit_card (id VARCHAR(20) PRIMARY KEY, iban VARCHAR(50) ...	0 row(s) affected

Inserción de datos:

```sql
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9564', 'XX6411797225628132080598 96', '3101008088023097', '1822', '7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9565', 'XX6428600219260557451637 58', '5522454509376387', '3053', '8
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9566', 'XX3270463329268112780556 30', '6161849983263025', '6889', '2
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9567', 'XX8563817838725386032800 64', '0745947204130257', '4955', '1
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9568', 'XX3865365478223814317476 59', '2296680186377148', '8326', '5
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9569', 'XX3052727289640478973615 63', '0734570924276022', '6083', '4
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9570', 'XX6339420102582861815165 73', '7233675965634598', '5619', '2
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9571', 'XX4849153824371419965794 68', '1265066247957405', '6585', '1
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9572', 'XX4054481862697303884045 08', '8296622848225053', '1995', '2
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9573', 'XX9653389310530889019249 06', '9099040179637171', '2277', '2
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9574', 'XX6627617583614326865207 75', '3171178331318656', '5084', '7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9575', 'XX4998812160736265711964 73', '8086033007377786', '7256', '6
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9576', 'XX5297109305877126417200 7', '4554226978064107', '4884', '72
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX1589140785948086331470 86', '8110437271856107', '5864', '7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX9915396464561105678702 54', '8999808823061411', '2872', '7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX2963930915871702021312 36', '9690060468678689', '8379', '1
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX7812588898519508066773 58', '5541182364498931', '9273', '7
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX9156705164053881243981 47', '2624305470167630', '4336', '9
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4999	14:35:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', '...	1 row(s) affected	0.000 sec
5000	14:35:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', '...	1 row(s) affected	0.000 sec

Limpieza de filas de la columna expiring_date:

```
60          -- Limpieza de filas de columna expiring_date:
61      -- 1. Desactivar el modo seguro
62      SET SQL_SAFE_UPDATES = 0;
63      -- 2. Ejecutar tu actualización masiva
64      UPDATE credit_card
65      SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
66      -- 3. Volver a activar el modo seguro por seguridad
67      SET SQL_SAFE_UPDATES = 1;
```

```
Output
Action Output
#  Time      Action                                                                                  Message                                                  Duration / Fetch
2  14:42:54  UPDATE credit_card  SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')         5000 row(s) affected Rows matched: 5000  Changed: 5000  Warnings: 0  0.094 sec
3  14:42:54  SET SQL_SAFE_UPDATES = 1                                                              0 row(s) affected                                        0.000 sec
```

Cambiar el tipo de dato en columna expiring_date a DATE:

```
68      -- cambiar el tipo de dato de la columna expiring_date a DATE
69      ALTER TABLE credit_card
70      MODIFY COLUMN expiring_date DATE;
```

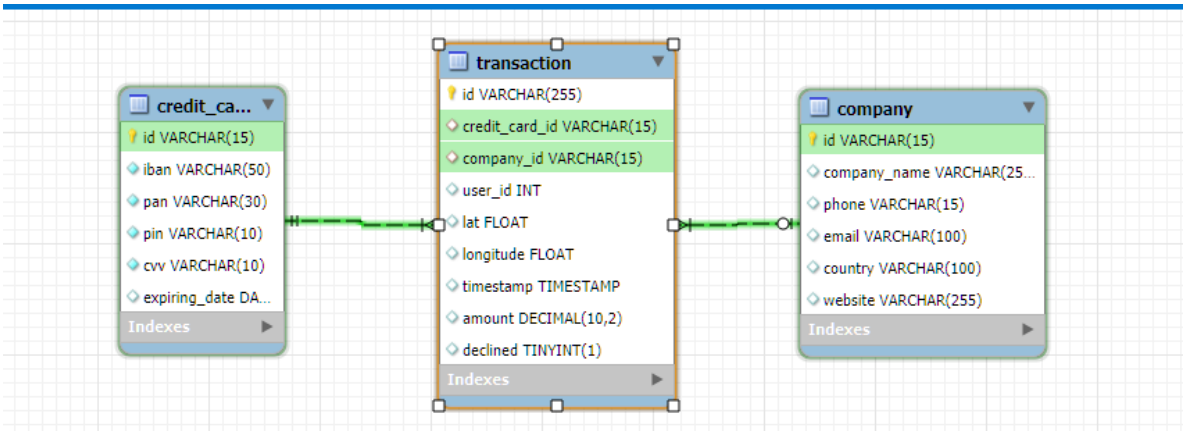
```
Output
Action Output
#  Time      Action                                                  Message                                                  Duration / Fetch
1  14:45:16  ALTER TABLE credit_card  MODIFY COLUMN expiring_date DATE  5000 row(s) affected Records: 5000  Duplicates: 0  Warnings: 0  0.094 sec
```

Relacionar tablas transaction con credit_card:

```
72      -- relacionar tablas transaction con credit_card:
73      ALTER TABLE transaction
74      ADD CONSTRAINT fk_transaction_credit_card
75      FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)
76      ON DELETE CASCADE
77      ON UPDATE CASCADE;
```

```
Output
Action Output
#  Time      Action                                                                  Message                                                    Duration / Fe
1  14:56:26  ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_card FOREIGN KE...  100000 row(s) affected Records: 100000  Duplicates: 0  Warnings: 0  2.281 sec
```

Recuerda mostrar el diagrama y realizar una breve descripción del mismo.



El diagrama representa la estructura de la base de datos transactions y las relaciones entre las tablas company, transaction y credit_card.

La tabla company almacena la información de las empresas y utiliza id como clave primaria para identificar de forma única cada empresa. La tabla credit_card almacena los datos de las tarjetas y utiliza id como clave primaria para identificar de forma única cada tarjeta.

La tabla transaction actúa como tabla principal de las operaciones y contiene las claves foráneas company_id y credit_card_id. company_id establece una relación de uno a muchos entre company y transaction, ya que una empresa puede realizar múltiples transacciones. Por otro lado, credit_card_id establece una relación de uno a muchos entre credit_card y transaction, ya que una tarjeta puede utilizarse en diferentes transacciones.

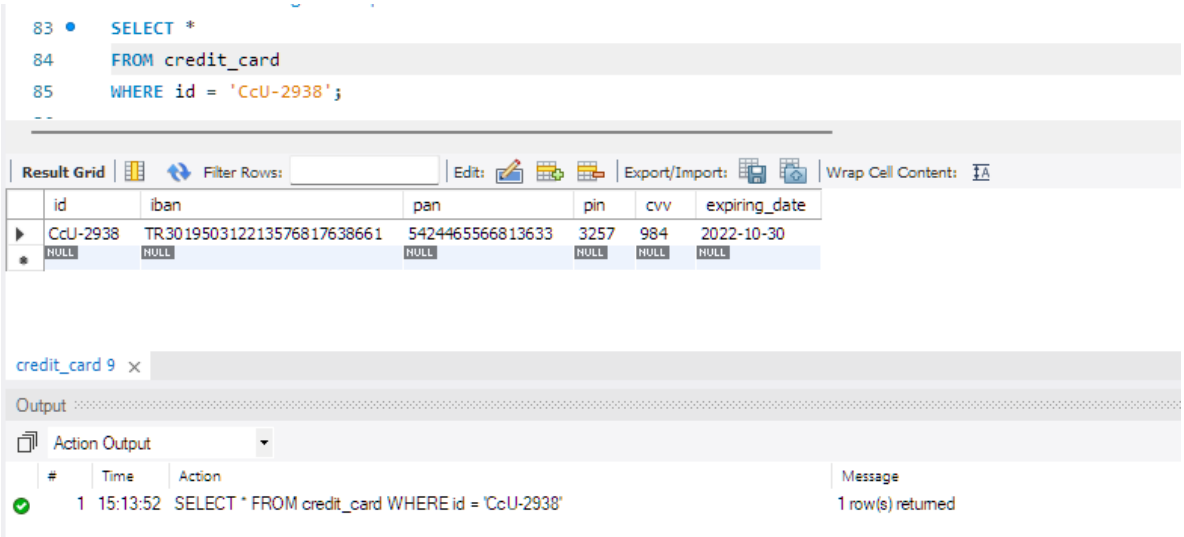
De esta forma, las claves primarias y foráneas permiten mantener la integridad referencial y relacionar correctamente las empresas, las tarjetas y las transacciones realizadas.

Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938.

La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

Revisión de registro que se hizo en ID CcU-2938:



Cambio de registro solicitado (TR323456312213576817699999):

```
87    -- Actualizar el número de cuenta (iban) de la tarjeta CcU-2938
88    UPDATE credit_card
89    SET iban = 'TR323456312213576817699999'
90    WHERE id = 'CcU-2938';
91
92
```

Output

Action Output

#	Time	Action	Message
1	15:15:44	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

Comprobación de cambio:

```
92    -- Comprobación de cambio:
93    SELECT *
94    FROM credit_card
95    WHERE id = 'CcU-2938';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 10

Output

Action Output

#	Time	Action	Message
1	15:17:43	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned

Ejercicio 6

En la tabla "transaction" ingresa una nueva transacción con la siguiente información: Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD credit_card_id CcU-9999 company_id b-9999 user_id 9999 lat 829.999 longitude -117.999 amount 111.11 declined 0

Primer paso: Inserción de datos completos a tabla company para evitar valores nulos, con el company_id b-9999

```
97    -- Primer paso: Insertar la compañía para evitar valores nulos y permitir la relación
98    INSERT INTO company (id, company_name, phone, email, country, website)
99    VALUES ('b-9999', 'Empresa Auxiliar HR', '+34000000000', 'contacto@b9999.com', 'España', 'www.b9999.com');
100
```

Output

Action Output

#	Time	Action	Message
1	15:25:07	INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-...	1 row(s) affected

Segundo paso: Inserción de datos completos en tabla credit_card para evitar valores nulos, con el credit_card_id CcU-9999

```sql
-- Insertar la tarjeta de crédito auxiliar para permitir la relación con la transacción
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date)
VALUES ('CcU-9999', 'TR99995031221357681763869 9', '5424465566813699', '1234', '123', '2028-12-31');
```

Output

Action Output

#	Time	Action	Message
1	15:29:28	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-9999', 'T...	1 row(s) affected

Tercer paso: Inserción de nuevo registro a transacciones con datos requeridos por el ejercicio.

```sql
-- Tercer paso inserción a tabla transaction
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, CURRENT_TIMESTAMP, 111.11, 0);
```

Output

Action Output

#	Time	Action	Message
1	15:29:28	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-9999', 'T...	1 row(s) affected
2	15:31:01	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestam...	1 row(s) affected

Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

```sql
-- Ejercicio 7
ALTER TABLE credit_card
DROP COLUMN pan;
DESCRIBE credit_card;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PRI	NULL	
iban	varchar(50)	NO		NULL	
pin	varchar(10)	NO		NULL	
cvv	varchar(10)	NO		NULL	
expiring_date	date	YES		NULL	

Result 11

Output

Action Output

#	Time	Action	Message
1	15:35:20	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	15:35:20	DESCRIBE credit_card	5 row(s) returned

Ejercicio 8
Descarga los archivos CSV que encontrarás en el apartado de recursos: american_users.csv, european_users.csv, companies.csv, credit_cards.csv, transactions.csv
Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Creación de Base de datos y su uso.

```
-- Ejercicio 8
-- Descarga los archivos CSV que encontrarás en el apartado de recursos :

-- american_users.csv
-- european_users.csv
-- companies.csv
-- credit_cards.csv
-- transactions.csv
-- Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consul
-- La tabla de products.csv la utilizaremos más adelante.

DROP DATABASE IF EXISTS transacciones_db;
CREATE DATABASE transacciones_db;
USE transacciones_db;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:53:59	DROP DATABASE IF EXISTS transacciones_db	14 row(s) affected	0.188 sec
2	16:53:59	CREATE DATABASE transacciones_db	1 row(s) affected	0.000 sec
3	16:53:59	USE transacciones_db	0 row(s) affected	0.000 sec

Habilitación para carga local de archivos CSV:

```
-- Habilitar carga local de archivos CSV
SET GLOBAL local_infile = 1;
```

Output

Action Output

#	Time	Action	Message
1	18:40:21	SET GLOBAL local_infile = 1	0 row(s) affected

Creación de tabla american_users:

```
-- Creación de tabla american_users
CREATE TABLE american_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date VARCHAR(20),
    country VARCHAR(50),
    city VARCHAR(50),
    postal_code VARCHAR(20),
    address VARCHAR(150),
    signup_date VARCHAR(20),
    user_segment VARCHAR(50),
    income_band VARCHAR(50)
);
```

Output

Action Output

#	Time	Action	Message
1	18:44:51	CREATE TABLE american_users (id INT PRIMARY KEY, name VARCHAR(100), ...	0 row(s) affected

Creación de tabla european_users:

```sql
-- Creación de tabla european_users
CREATE TABLE european_users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date VARCHAR(20),
    country VARCHAR(50),
    city VARCHAR(50),
    postal_code VARCHAR(20),
    address VARCHAR(150),
    signup_date VARCHAR(20),
    user_segment VARCHAR(50),
    income_band VARCHAR(50)
);
```

Output

Action Output

#	Time	Action	Message
1	18:46:37	CREATE TABLE european_users (id INT PRIMARY KEY, name VARCHAR(100), ...	0 row(s) affected

Creación de tabla companies

```sql
-- Creación de tabla companies
CREATE TABLE companies (
    company_id VARCHAR(50) PRIMARY KEY,
    company_name VARCHAR(150),
    phone VARCHAR(50),
    email VARCHAR(100),
    country VARCHAR(50),
    website VARCHAR(150),
    merchant_category VARCHAR(100),
    merchant_price_position VARCHAR(50)
);
```

Output

Action Output

#	Time	Action	Message
1	18:50:38	CREATE TABLE companies (company_id VARCHAR(50) PRIMARY KEY, company_...	0 row(s) affected

Creación de tabla credit_cards

```sql
-- Creación de tabla credit_cards
CREATE TABLE credit_cards (
    id VARCHAR(50) PRIMARY KEY,
    user_id VARCHAR(50),
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    track1 TEXT,
    track2 TEXT,
    exping_date VARCHAR(20),
    card_type VARCHAR(50),
    card_renewal_flag VARCHAR(10)
);
```

Output

Action Output

#	Time	Action	Message
1	19:05:47	CREATE TABLE credit_cards (id VARCHAR(50) PRIMARY KEY, user_id VARCHAR...	0 row(s) affected

Creación de tabla transactions

```sql
DROP TABLE IF EXISTS transactions;
CREATE TABLE transactions (
    id VARCHAR(50) PRIMARY KEY,
    card_id VARCHAR(50),
    business_id VARCHAR(50),
    timestamp DATETIME,
    amount DECIMAL(12,4),
    declined BOOLEAN,
    product_ids TEXT,
    user_id INT,
    lat DOUBLE,
    longitude DOUBLE,
    discount_amount DECIMAL(12,4),
    tax_amount DECIMAL(12,4),
    shipping_amount DECIMAL(12,4),
    channel VARCHAR(50),
    campaign_id VARCHAR(50),
    device_type VARCHAR(50),
    is_international BOOLEAN,
    decline_reason VARCHAR(100),
```

Output

Action Output

#	Time	Action	Message
1	18:10:29	CREATE TABLE transactions (id VARCHAR(50) PRIMARY KEY, card_id VARCHAR...	0 row(s) affected

Cargar los datos desde archivos CSV:

Cargando datos a tabla american_users

```
218 •  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__american_users.csv'
219    IGNORE
220    INTO TABLE american_users
221    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
222    LINES TERMINATED BY '\n'
223    IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message	Duration / Fe
1	19:15:56	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec

Cargando datos en tablas european_users

```
224
225 •  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__european_users.csv'
226    IGNORE
227    INTO TABLE european_users
228    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
229    LINES TERMINATED BY '\n'
230    IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:18:01	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.109 sec

Cargando datos a tabla companies

```
232 •  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__companies.csv'
233    IGNORE
234    INTO TABLE companies
235    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
236    LINES TERMINATED BY '\n'
237    IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
1	19:25:26	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Cargando datos a tabla credit_cards

```
239 •  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__credit_cards.csv'
240    INTO TABLE credit_cards
241    FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
242    LINES TERMINATED BY '\n'
243    IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
1	19:26:37	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Cargando datos a tabla transactions:

```
246 •  TRUNCATE transactions;
247 •  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__transactions.csv'
248    INTO TABLE transactions
249    FIELDS TERMINATED BY ';' OPTIONALLY ENCLOSED BY '"'
250    LINES TERMINATED BY '\n'
251    IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	18:12:54	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	4.843 sec

Modelo estrella, creación y unificación de tabla de dimensiones dim_user con columna de macro_region para diferenciar el continente del usuario.

```
255 •  DROP TABLE IF EXISTS dim_user;
256 •  CREATE TABLE dim_user AS
257    SELECT
258        CAST(id AS CHAR(50)) AS user_id,
259        'Americas' AS macro_region,
260        name, surname, phone, email, birth_date, country, city, postal_code, address
261    FROM american_users
262
263    UNION ALL
264
265    SELECT
266        CAST(id AS CHAR(50)) AS user_id,
267        'Europe' AS macro_region,
268        name, surname, phone, email, birth_date, country, city, postal_code, address
269    FROM european_users;
```

Output

Action Output

#	Time	Action	Message	Duration / F
1	20:55:59	DROP TABLE IF EXISTS dim_user	0 row(s) affected	0.015 sec
2	20:55:59	CREATE TABLE dim_user AS SELECT CAST(id AS CHAR(50)) AS user_id, ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.125 sec

Añadiendo PK a dim_user

```
268
269 •  ALTER TABLE dim_user ADD PRIMARY KEY (user_id);
```

Output

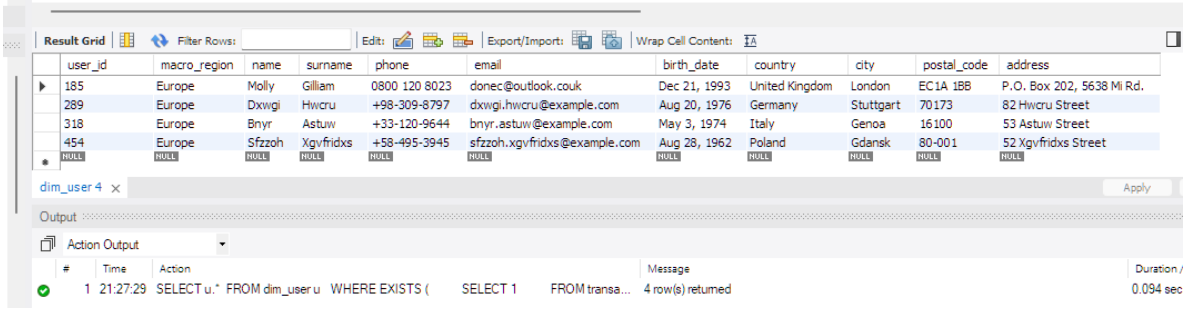
Action Output

#	Time	Action	Message	Duration / Fetc
1	19:59:43	ALTER TABLE dim_user ADD PRIMARY KEY (user_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec

Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones usando al menos 2 tablas.

```sql
-- Ejercicio 9 (Nivel 1): Usuarios con más de 80 transacciones
SELECT u.*
FROM dim_user u
WHERE EXISTS (
    SELECT 1
    FROM transactions t
    WHERE t.user_id = u.user_id
    GROUP BY t.user_id
    HAVING COUNT(t.id) > 80
);
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

user_id	macro_region	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Europe	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.
289	Europe	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street
318	Europe	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street
454	Europe	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk	80-001	52 Xgvfridxs Street
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

dim_user 4 | Apply

Output

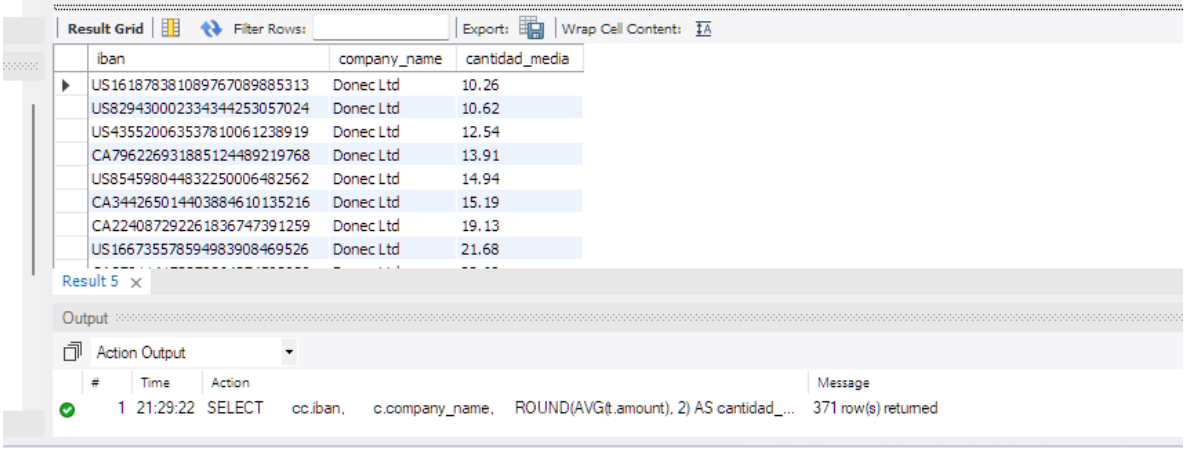
Action Output

#	Time	Action	Message	Duration /
1	21:27:29	SELECT u.* FROM dim_user u WHERE EXISTS (SELECT 1 FROM transa...	4 row(s) returned	0.094 sec

Ejercicio 10

Muestra la cantidad media por IBAN para tarjetas de crédito en la empresa Donec Ltd, utiliza al menos 2 tablas.

```sql
-- Ejercicio 10 (Nivel 1): Cantidad media por IBAN para Donec Ltd
SELECT
    cc.iban,
    c.company_name,
    ROUND(AVG(t.amount), 2) AS cantidad_media
FROM transactions t
INNER JOIN credit_cards cc ON t.card_id = cc.id
INNER JOIN companies c ON t.business_id = c.company_id
WHERE c.company_name = 'Donec Ltd'
GROUP BY cc.iban
ORDER BY cantidad_media;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	company_name	cantidad_media
US16187838108976708988531 3	Donec Ltd	10.26
US82943000233434425305702 4	Donec Ltd	10.62
US43552006353781006123891 9	Donec Ltd	12.54
CA79622693188512448921976 8	Donec Ltd	13.91
US85459804483225000648256 2	Donec Ltd	14.94
CA34426501440388461013521 6	Donec Ltd	15.19
CA22408729226183674739125 9	Donec Ltd	19.13
US16673557859498390846952 6	Donec Ltd	21.68

Result 5

Output

Action Output

#	Time	Action	Message
1	21:29:22	SELECT cc.iban, c.company_name, ROUND(AVG(t.amount), 2) AS cantidad_...	371 row(s) returned

Nivel 2. Ejercicio 1: Identifica los cinco días en los que se generó la mayor cantidad de ingresos en la empresa a partir de las ventas. Muestra la fecha de cada transacción junto con el total de ventas.

```sql
-- Ejercicio 1 (Nivel 2): 5 días con mayores ingresos por ventas
SELECT
    DATE(t.timestamp) AS fecha_venta,
    SUM(t.amount) AS total_ingresos
FROM transactions t
WHERE t.declined = 0
GROUP BY fecha_venta
ORDER BY total_ingresos DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fecha_venta	total_ingresos
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Result 6

Output

Action Output

#	Time	Action		Message
1	21:35:15	SELECT	DATE(t.timestamp) AS fecha_venta,	SUM(t.amount) AS total_ingresos F... 5 row(s) returned

Ejercicio 2

Presentar el nombre, número de teléfono, país, fecha y importe, de las empresas que realizaron transacciones con un valor entre 350 y 400 euros y en cualquiera de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor.

```sql
-- Ejercicio 2 (Nivel 2): Transacciones entre 350 y 400€ en fechas específicas
SELECT
    c.company_name,
    c.phone,
    c.country,
    DATE(t.timestamp) AS fecha,
    t.amount
FROM transactions t
INNER JOIN companies c ON t.business_id = c.company_id
WHERE t.declined = 0
  AND t.amount BETWEEN 350 AND 400
  AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
ORDER BY t.amount DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	fecha	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21

Result 11

Output

Action Output

#	Time	Action		Message
1	21:49:45	SELECT	c.company_name, c.phone, c.country, DATE(t.timestamp) AS fe...	6 row(s) returned

Ejercicio 3: Necesitamos optimizar la asignación de recursos y dependerá de la capacidad operativa que se requiera, así que te piden información sobre el número de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere una lista de empresas donde especifiques si tienen igual o más de 400 transacciones o menos.

```
-- Ejercicio 3 (Nivel 2): Categoría operativa por transacciones (>=400 o menos)
SELECT
    c.company_name,
    COUNT(t.id) AS total_transacciones,
    CASE
        WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400 transacciones'
        ELSE 'Menos de 400 transacciones'
    END AS categoria_operativa
FROM companies c
LEFT JOIN transactions t ON c.company_id = t.business_id
GROUP BY c.company_id
ORDER BY total_transacciones DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	total_transacciones	categoria_operativa
Ac Fermentum Incorporated	2401	Igual o más de 400 transacciones
Nunc Interdum Incorporated	1599	Igual o más de 400 transacciones
Donec Fringilla PC	1593	Igual o más de 400 transacciones
Mauris Institute	1586	Igual o más de 400 transacciones
Rutrum Non Inc.	1585	Igual o más de 400 transacciones
Aliquet Vel Vulputate Incorporated	1583	Igual o más de 400 transacciones
Aliquam PC	1576	Igual o más de 400 transacciones

Result 10

Output

Action Output

#	Time	Action	Message
1	21:45:39	SELECT c.company_name, COUNT(t.id) AS total_transacciones, CASE ...	100 row(s) returned

Ejercicio 4
Elimina de la tabla de transacciones el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
Primer paso verificar registro antes de ser borrado de la base de datos:

```
-- Ejercicio 4 (Nivel 2): Eliminar registro específico de la tabla de hechos
-- Verificación de registro antes de borrar:
SELECT * FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discour
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.59720554463741	12.22175994259365	19.92

transactions 12

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:04:31	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CA...	1 row(s) returned	0.000 sec / 0.0

Segundo paso eliminar el registro en tabla

```
343     -- ELiminación de registro
344 •   DELETE FROM transactions
345     WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Output

Action Output

#	Time	Action	Message
1	22:09:00	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Ejercicio 5

La sección de marketing quiere tener acceso a información específica para llevar a cabo análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las empresas y sus transacciones. Tendrás que crear una vista llamada VistaMarketing que contenga la siguiente información: nombre de la empresa. Contacta con el número de teléfono. País de residencia. Compra media realizada por cada empresa. Presenta la vista creada, ordenando los datos de la compra media más alta a la media más baja.

Primer paso creación de vista:

```
347     -- Ejercicio 5 (Nivel 2): Vista VistaMarketing
348 •   CREATE OR REPLACE VIEW VistaMarketing AS
349     SELECT
350         c.company_name AS nombre_compañía,
351         c.phone AS teléfono_contacto,
352         c.country AS país_residencia,
353         ROUND(AVG(t.amount), 2) AS media_compra
354     FROM companies c
355     LEFT JOIN transactions t ON c.company_id = t.business_id
356     GROUP BY c.company_id;
357
```

Output

Action Output

#	Time	Action	Message
1	22:14:08	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS nom...	0 row(s) affected

Segundo paso, muestra de vista creada:

```
357
358    -- Consulta para ver el resultado de la vista
359 •  SELECT * FROM VistaMarketing;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre_compañía	teléfono_contacto	país_residencia	media_compra
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.77
Magna A Neque Industries	+33 823 560 3779	Australia	268.44
Fusce Corp.	+29 657 646 4324	United States	274.28
Convallis In Incorporated	+6 693 982 1255	Germany	298.90
Ante Iaculis Nec Foundation	+69 914 549 1009	New Zealand	268.32
Donec Ltd	+56 897 341 8418	Norway	272.76
Sed Nunc Ltd	+5 218 900 3193	United Kingdom	295.24
Amet Nulla Donec Corporation	+96 338 276 8449	Italy	299.50
Nascetur Ridiculus Mus Inc.	+63 316 953 7970	United States	271.83
Vestibulum Lorem PC	+27 788 560 5000	Belgium	273.72
Gravida Sagittis LLP	+98 204 198 9702	Sweden	297.95

VistaMarketing 14 ×

Output

Action Output

#	Time	Action	Message
1	22:15:29	SELECT * FROM VistaMarketing	100 row(s) returned

Nivel 3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si por lo menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

Paso 1: Crear la tabla temporal o definitiva con el estado de cada tarjeta

```
364 •  CREATE TABLE card_status AS
365    WITH ranked_transactions AS (
366        SELECT
367            card_id,
368            declined,
369            ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS rn
370        FROM transactions
371    ),
372    last_three AS (
373        SELECT card_id, declined
374        FROM ranked_transactions
375        WHERE rn <= 3
376    )
377    SELECT
378        card_id,
379        CASE
380            WHEN SUM(declined) = 3 THEN 'inactivo'
381            ELSE 'activo'
382        END AS status
383    FROM last_three
384    GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message
1	22:26:56	CREATE TABLE card_status AS WITH ranked_transactions AS (SELECT card_i...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

¿Cuántas tarjetas están activas?

```sql
-- Cuántas tarjetas están activas?
SELECT COUNT(*) AS tarjetas_activas
FROM card_status
WHERE status = 'activo';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tarjetas_activas
5000

Result 15

Output

Action Output

#	Time	Action	Message
1	22:30:07	SELECT COUNT(*) AS tarjetas_activas FROM card_status WHERE status = 'activo'	1 row(s) returned

Ejercicio 2

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:

Creación de tabla

```sql
DROP TABLE IF EXISTS dim_product;
CREATE TABLE dim_product (
    product_id VARCHAR(50) PRIMARY KEY,
    product_name VARCHAR(255),
    price DECIMAL(12,4),
    colour VARCHAR(50),
    weight DECIMAL(12,4),
    warehouse_id VARCHAR(50),
    category VARCHAR(100),
    brand VARCHAR(100),
    cost DECIMAL(12,4),
    launch_date DATE
);
```

Output

Action Output

#	Time	Action	Message
1	18:16:12	DROP TABLE IF EXISTS dim_product	0 row(s) affected
2	18:16:22	CREATE TABLE dim_product (product_id VARCHAR(50) PRIMARY KEY, product_n...	0 row(s) affected

Agregación de columna: "currency" para no perder datos existentes y de la misma forma limpiar las columnas “price” y “cost” para poder hacer operaciones. Todos los campos de tipo precio y coste en las tablas del sistema están expresados en dólares estadounidenses (USD [$]).

```
589  -- Agregación de columna: "currency" para no perder datos existentes y de la misma forma limpar las columnas
590  -- price y cost para poder hacer operaciones.
591  ALTER TABLE dim_product ADD COLUMN currency VARCHAR(5) DEFAULT 'USD';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:22:02	ALTER TABLE dim_product ADD COLUMN currency VARCHAR(5) DEFAULT 'USD'	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec

Inserción de los datos a la tabla dim_product

```
413  -- Inserción de datos a dim_product
414
415  LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/ARchivosEjercicio8/N1-Ex.8__products.csv'
416  INTO TABLE dim_product
417  CHARACTER SET utf8mb4
418  FIELDS TERMINATED BY ','
419  OPTIONALLY ENCLOSED BY '"'
420  LINES TERMINATED BY '\n'
421  IGNORE 1 ROWS
422  (product_id, product_name, @price, colour, @weight, warehouse_id, category, brand, @cost, launch_date)
423  SET
424      price = CAST(REPLACE(@price, '$', '') AS DECIMAL(12,4)),
425      weight = CAST(@weight AS DECIMAL),
426      cost = CAST(REPLACE(@cost, '$', '') AS DECIMAL(12,4)),
427      currency = 'USD';
428
```

Output

Action Output

#	Time	Action	Message	Duration / F
1	18:19:34	LOAD DATA LOCAL INFILE 'C:/Users/freak/OneDrive/Documentos/ITAcademy/sprint1/...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.015 sec

Normalización de N:M (Tabla intermedia de productos)

```
414  CREATE TABLE transaction_products (
415      transaction_id VARCHAR(255),
416      product_id VARCHAR(50),
417      PRIMARY KEY (transaction_id, product_id),
418      CONSTRAINT fk_tp_transaction FOREIGN KEY (transaction_id) REFERENCES transactions(id),
419      CONSTRAINT fk_tp_product FOREIGN KEY (product_id) REFERENCES dim_product(product_id)
420  );
```

Output

Action Output

#	Time	Action	Message
1	16:41:41	CREATE TABLE transaction_products (transaction_id VARCHAR(255), product_id V...	0 row(s) affected

Poblar la tabla puente transaction_products

```sql
-- Poblar la tabla puente desglosando la lista delimitada por comas
INSERT INTO transaction_products (transaction_id, product_id)
SELECT DISTINCT
    t.id,
    TRIM(j.product_id) AS product_id
FROM transactions t
JOIN JSON_TABLE(
    CONCAT('["', REPLACE(t.product_ids, ',', '","'), '"]'),
    '$[*]' COLUMNS (product_id VARCHAR(50) PATH '$')
) j
WHERE t.product_ids IS NOT NULL AND t.product_ids != '';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:06:48	INSERT INTO transaction_products (transaction_id, product_id) SELECT DISTINCT t.i...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	12.047 sec

Eliminación de columna multivaluada de transactions:

```sql
-- Una vez normalizado, eliminamos la columna multivaluada de transactions
ALTER TABLE transactions DROP COLUMN product_ids;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:46:41	ALTER TABLE transactions DROP COLUMN product_ids	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.078 sec

Consulta para conocer el número de veces que se ha vendido cada producto

```sql
-- Ejercicio 2 (Nivel 3): Número de ventas por producto (Usando la tabla intermedia)
SELECT
    p.product_id,
    p.product_name,
    COUNT(tp.transaction_id) AS total_veces_vendido
FROM dim_product p
LEFT JOIN transaction_products tp ON p.product_id = tp.product_id
LEFT JOIN transactions t ON tp.transaction_id = t.id AND t.declined = 0
GROUP BY p.product_id
ORDER BY total_veces_vendido DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	total_veces_vendido
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2601

Result 5

Output

Action Output

#	Time	Action	Message
1	18:24:43	SELECT p.product_id, p.product_name, COUNT(tp.transaction_id) AS total_vec...	100 row(s) returned

Enlazar FK con PK faltantes para realizar el modelo estrella:

```
500     -- 1. Enlazar transactions con la dimensión companies (Empresas)
501 •   ALTER TABLE transactions
502     ADD CONSTRAINT fk_transactions_company
503     FOREIGN KEY (business_id) REFERENCES companies(company_id)
504     ON DELETE CASCADE
505     ON UPDATE CASCADE;
506
```

Output

Action Output

#	Time	Action	Message
1	12:22:15	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_company FOREIGN KE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Enlazar transactions con la dimensión dim_user

```
507     -- 2. Enlazar transactions con la dimensión dim_user (Usuarios)
508 •   ALTER TABLE transactions
509     ADD CONSTRAINT fk_transactions_user
510     FOREIGN KEY (user_id) REFERENCES dim_user(user_id)
511     ON DELETE CASCADE
512     ON UPDATE CASCADE;
513
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:25:06	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_user FOREIGN KEY (us...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.438 sec

Enlazar transactions con credit_cards

```
514     -- 3. Enlazar transactions con credit_cards (Tarjetas de crédito)
515 •   ALTER TABLE transactions
516     ADD CONSTRAINT fk_transactions_card
517     FOREIGN KEY (card_id) REFERENCES credit_cards(id)
518     ON DELETE CASCADE
519     ON UPDATE CASCADE;
520
```

Output

Action Output

#	Time	Action	Message
1	12:26:36	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_card FOREIGN KEY (c...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Enlazar card_status con credit_cards

```
521     -- 4. Enlazar card_status con credit_cards (según tu requerimiento)
522 •   ALTER TABLE card_status
523     ADD CONSTRAINT fk_card_status_card
524     FOREIGN KEY (card_id) REFERENCES credit_cards(id)
525     ON DELETE CASCADE
526     ON UPDATE CASCADE;
```

Output

Action Output

#	Time	Action	Message
1	12:48:01	ALTER TABLE card_status ADD CONSTRAINT fk_card_status_card FOREIGN KEY (ca...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Diagrama Estrella:

